

Running Custom Model Evaluations (Text-to-SQL Case Study)

1. Core Framework of LLM Evaluations

In Large Language Model (LLM) evaluations, we establish a foundational distinction between the two primary paradigms of testing. Before we can build, deploy, or optimize any AI application, we must understand how to measure its performance.

Model Evaluations vs. Application Evaluations

- **Model Evaluations (Model Evals):** This paradigm focuses on evaluating the raw capabilities of the underlying LLM itself. We test the model as an isolated engine to understand its boundaries, strengths, and weaknesses across various domains. This is the stage where we answer: "Which base model is inherently better at reasoning, mathematics, or coding?"
- **Application Evaluations (Application Evals):** This paradigm focuses on evaluating the entire pipeline of our specific AI application, which includes the retriever, the prompt templates, the guardrails, and the model's outputs in a multi-step workflow. While our main focus in this course is on Application Evals, we must thoroughly cover Model Evals first, as selecting the right model is the prerequisite to building a robust application.

Generic Benchmarks vs. Custom Model Evaluations

Within the domain of Model Evals, we split our methodology into two distinct approaches:

1. **Generic Benchmarks:** These are standardized, publicly available datasets (e.g., MMLU, GSM8K, HumanEval) designed to test specific, isolated LLM capabilities such as general knowledge, logical reasoning, mathematics, and code generation. While benchmarks are invaluable for narrowing down the landscape of available models to a few top candidates, they fail to tell us how a model will perform on our proprietary data.
2. **Custom Model Evaluations (Custom Evals):** This is the practice of evaluating candidate LLMs on our own domain specific data and tasks. To make an informed engineering and business decision, we must run the candidate models through our actual system prompts, using our actual database schemas and user queries.

2. Case Study: Text-to-SQL for Live Cricket (Ask Cricinfo)

To master the practical execution of a custom model evaluation, we will place ourselves in the shoes of **AI Engineers** working for **ESPN Cricinfo**, one of the world's leading platforms for live cricket scores and analysis.

The Problem Statement

During live cricket matches (especially massive high-stakes matches like India vs. Pakistan or CSK vs. MI), the volume of user queries spikes exponentially. Users follow the text commentary feed and constantly want answers to highly specific, contextual historical questions, such as:

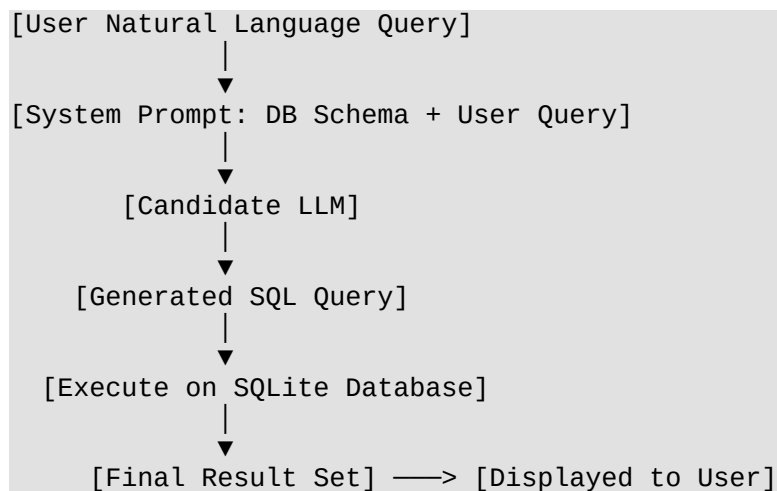
- “What is Virat Kohli's batting average against Pakistan?”
- “How many wickets did Jaspreet Bumrah take the last time India played Pakistan?”

Traditionally, Cricinfo managed this using a manual loop: $\text{Commentary Team} \rightarrow \text{Data Analysts} \rightarrow \text{Database Query (SQL)} \rightarrow \text{Result to Feed}$

An analyst team with direct database access would receive questions from the commentators, write SQL queries on the fly, run them, and send the answers back to be published. This process is **completely unscalable**. During peak hours, they can receive up to 500 questions in 5 minutes, making it impossible for human analysts to keep up.

The Solution Architecture (Text-to-SQL)

To solve this scalability bottleneck, we must build a feature called “**Ask Cricinfo**” a scalable, automated **Text-to-SQL system** where an LLM acts as the central brain:



Why This is Not a RAG Pipeline

It is critical to note that **this is not a Retrieval Augmented Generation (RAG) pipeline**. In a typical RAG system, we retrieve dynamic external document chunks for every query. Here, we do not provide dynamic documents. Instead, we provide a **static database schema** that remains identical across all queries. The model's sole job is to translate natural language into structured SQLite syntax based on that schema.

3. Stage 1: Detailed Requirements Gathering

Every successful AI engineering project must begin with an exhaustive requirements gathering phase. We do not select models based on personal preferences or general market hype, we select them based on rigorous technical and financial metrics.

Key Requirement Dimensions

1. Task Identification

We are explicitly building a **Text-to-SQL engine** operating within a narrow, focused domain: the **Indian Premier League (IPL)**. This narrowing of the domain allows us to optimize performance and prevent out-of-scope hallucinations.

2. Cost Ceiling and Business ROI

Any feature we build must make business sense. For a global platform like ESPN Cricinfo, providing an interactive query feature is a **User Retention Play**.

- **The Business Loop:** Higher retention $\rightarrow$ longer time spent on the website $\rightarrow$ more ad impressions served $\rightarrow$ increased revenue.
- **Budget Guideline:** In my discussions with our business managers, we have established a strict **monthly budget ceiling of 3 Lakhs INR** (though for our initial technical preparations, we can model up to 5 Lakhs INR to see what premium options might look like). If a model's running costs exceed this ceiling, it is immediately disqualified.

3. Detailed Cost Calculation Methodology

We must mathematically estimate our monthly operating costs before running evaluations. LLM providers charge us on a **per token basis**, with different rates for input and output tokens.

Let us define our baseline variables:

- **Input Prompt Size:** Our system prompt (containing instructions and the complete database schema) plus the user question averages **400 tokens**.
- **Output Size:** Since we require only the raw SQL query with no conversational explanation, our output averages **100 tokens**.
- **Input-to-Output Ratio:** This gives us an exact **4:1 ratio** (400 input tokens to 100 output tokens).
- **Daily Query Volume:** We assume an average of **50,000 (50k) queries per day**. While traffic is highly spiky (peaking during major matches and dropping at night), 50k is our calculated daily average.
- **USD to INR Conversion Rate:** We use a conversion factor of **95** to account for taxes, platform fees, and exchange rates.

Let us run a sample calculation using a premium frontier model, **Claude 3.5 Fable** (fictional premium

model for calculations), which charges:

- **Input Cost (\$C_{in}):** \$10 per Million (1M) tokens
- **Output Cost (\$C_{out}):** \$50 per Million (1M) tokens

$$\text{Cost per Query} = \left(\frac{\$10 \times 400}{1,000,000}\right) + \left(\frac{\$50 \times 100}{1,000,000}\right)$$
$$\text{Cost per Query} = \$0.004 + \$0.005 = \$0.009 \text{ USD}$$

$$\text{Daily Operating Cost} = 50,000 \text{ queries} \times \$0.009 = \$450 \text{ USD/day}$$
$$\text{Monthly Operating Cost (USD)} = \$450 \times 30 \text{ days} = \$13,500 \text{ USD/month}$$
$$\text{Monthly Operating Cost (INR)} = \$13,500 \times 95 = \mathbf{1,282,500 \text{ INR/month}}$$

(~12.82 Lakhs)

At **12.82 Lakhs INR/month**, Claude 3.5 Fable exceeds our 3 Lakhs budget by over **4x**. This mathematical reality check tells us that we must explore more cost effective models, or leverage advanced optimizations.

4. Optimizing Costs via Prompt Caching (KV Caching)

To combat high token costs, we can leverage **Prompt Caching** (internally powered by **Key-Value or KV Caching** in the attention mechanism of transformers). Because our database schema is static and constitutes the bulk of our 400 token input, we can cache it on the LLM provider's servers.

- **How KV Caching Works:** The model caches the key and value vectors of the static prompt segment. When a new query arrives, the model only processes the new user query tokens from scratch and reads the rest from the high speed cache.
- **Pricing Dynamics (Example):**
 - **Cache Write / Processing Cost:** \$12.50 per Million tokens (slightly higher than the normal \$10 rate to establish the cache).
 - **Cache Hit Cost:** \$1 per Million tokens (a **10x reduction** compared to the \$10 base rate).
- **Refresh Cycles:**
 - **5-Minute Cache TTL (Time-to-Live):** If a query is received within 5 minutes of the last one, it hits the cache. For Cricinfo's 50k daily queries, a question comes in every few seconds, making a 5-minute TTL highly efficient.
 - **1-Hour Cache TTL:** Suitable for low-traffic applications (e.g., 1,000 queries per day) where queries are spaced far apart.
- **RAG vs. Text-to-SQL Caching:** Prompt caching is highly effective for Text-to-SQL because the DB schema is static. In contrast, it is highly ineffective for RAG chatbots because the retrieved context blocks change dynamically with every user turn, constantly invalidating the cache.
- **Savings Potential:** Implementing prompt caching can slash our monthly input costs by up to **40% to 50%**, potentially dropping a 12.8 Lakhs bill down to **6–7 Lakhs**.

5. Latency

During a live match, user curiosity is fleeting. If our system takes too long, it triggers user anxiety and ruins the experience.

- **Target Latency:** We require a response time of **2 to 3 seconds**. Any model requiring extensive "thinking" or reasoning steps that push latency beyond 5 seconds is highly undesirable.

6. Context Window

Since each user query is independent, disjoint, and single-turn, there is no multi turn session history to maintain. Therefore, **context window size is not a limiting constraint** for this application; even a model with a very small context window is perfectly acceptable.

7. Deployment & Infrastructure

We explicitly favor **Public APIs** (e.g., OpenAI, Anthropic, xAI) over self hosting open source models. Public APIs offer mature, robust, and highly reliable infrastructure. Setting up on-premises servers to host large models requires massive technical overhead and constant maintenance, which is highly unscalable and prone to downtime.

8. Correctness and Brand Safety

Our users are passionate cricket fans who are incredibly finicky about statistics and historical records.

- **The Risk:** If the model generates an incorrect SQL query that yields wrong statistics (e.g., listing a bowler as the highest run scorer), users will immediately screenshot the error and post it on Twitter or Instagram, causing massive public embarrassment and damaging ESPN Cricinfo's brand reputation. **Correctness is our absolute, non-negotiable metric.**
-

4. Stage 2: Leaderboard Consultation and Shortlisting

With our technical requirements defined, we proceed to Stage 2: utilizing public LLM leaderboards to filter and shortlist our candidate models. Remember: **Leaderboards are for filtration, not final selection.**

Critiquing Specialized Text-to-SQL Leaderboards

We initially look for specialized Text-to-SQL benchmarks, but find them highly problematic:

1. **BIRD SQL Benchmark:** This is a well known benchmark, but I rejected it for two reasons. First, it is outdated and fails to include the newest generation of frontier models. Instead, it features highly specialized, obscure, fine-tuned models that are not easily accessible for production. Second, I strongly dislike their naming convention (deriving "BIRD" by arbitrarily extracting letters from "Big" and "Large").
2. **LiveSQLBench:** While more active, it is still not updated frequently enough to include the absolute latest model versions.

Using Coding Leaderboards as a Proxy

Since generating SQL is fundamentally a code generation task, we can use general **Coding Leaderboards** as a proxy (alias) for a model's SQL generation capabilities.

We consult **LLMstats.com**, a premier third-party aggregator that compiles multiple coding benchmarks, remains highly up-to-date with the latest models, and provides critical metadata such as model speed (characters per second) and pricing.

Understanding Blended Pricing

Leaderboards often display a single **Blended Price** per Million tokens instead of separating input and output costs. To interpret this correctly, we must look at the blending ratio.

- **LLMstats Ratio:** Uses a **4:1 blended ratio** (4 parts input, 1 part output).
- **Calculation Example:** For Claude 3.5 Fable, with an input cost of \$10 and an output cost of \$50:

$$\text{Blended Price} = \frac{(4 \times \$10) + (1 \times \$50)}{4 + 1} = \frac{\$40 + \$50}{5} = \$18 \text{ per Million tokens}$$

Because our Cricinfo application has an actual input-to-output ratio of exactly **400 to 100 (4:1)**, this blended price is **perfectly tailored to our application's cost modeling**. We can directly use this single blended price to project our monthly costs for all 146 models listed on the leaderboard.

Normalization and Weighted Scoring

To rank the models within our budget, we download the dataset of 146 models and apply **Min-Max Normalization** to bring two key metrics—**Coding Capability (Rating)** and **Speed (Characters per Second)**—into a shared **0 to 1 scale**:

$$\text{Normalized Score} = \frac{\text{Value} - \text{Min}}{\text{Max} - \text{Min}}$$

We then calculate a combined score for every model using a **Weighted Sum Formula**:

$$\text{Combined Score} = (0.9 \times \text{Normalized Rating}) + (0.1 \times \text{Normalized Speed})$$

- **Why 90% Weight on Rating & 10% on Speed?** Because our output SQL query is very short (~100 tokens), even a slower model will print it in under two seconds. Correctness is infinitely more important than raw generation speed in a Text-to-SQL task, so we heavily penalize latency compromises in favor of accuracy.

Shortlisting the Candidate Models

After filtering out all models that exceed our monthly budget ceiling, we shortlist the following top candidates for our custom evaluation:

1. **GPT 5.6 Tera:** The absolute top performing model from OpenAI. Highly capable but sits at the upper limit of our budget.

2. **Kimi K3 (Km K3)**: A highly hyped 2.7-Trillion parameter model from Moonshot AI. It offers frontier level performance on paper at 1/3 of the cost of GPT Tera.
3. **Grok**: An extremely fast, highly underrated model from xAI. It represents a very cost effective and powerful alternative.
4. **Claude 3.5 Sonnet 5**: The flagship representative of the Anthropic family, renowned for its stability and coding precision.
5. **MiniMax M3**: A highly efficient, low-cost Chinese model that performs surprisingly well on general coding benchmarks.

*(Note: Meta's **Muse Spark 1.1** was ranked near the top, but we had to skip it because it is currently restricted to US users and unavailable for our deployment. We also skipped **GPT 5.6 Luna** because it is a smaller variant from the same family as Tera, and we want to test the absolute best representative of that family).*

5. Stage 3: Running the Custom Model Evaluation

Now we execute the core technical process: running our candidate models against our custom data and evaluating their output.

The Ingredients Needed

1. The Database

We download the complete **IPL historical dataset (2008–2024)** from Kaggle. To keep our evaluation fast and efficient during our live execution, we trim this dataset to focus exclusively on the years **2020 to 2024**. We store this as two CSV files:

- `matches.csv`: Contains metadata for every match (teams, venue, date, umpires, toss decisions, etc.).
- `deliveries.csv`: Contains granular, ball-by-ball records for every single delivery (batsman, bowler, runs scored, extras, wicket types, fielder names, etc.).

We write a database setup script (`db.py`) to read these CSVs and load them into a **SQLite database**.

2. The Database Schema

We write a schema extractor script (`schema_extractor.py`) to systematically read our SQLite database and generate a clean, structured schema file (`schema.sql`).

This schema is vital because it will be dynamically injected into the system prompt of every candidate model, ensuring the LLM knows the exact table names, column names, and data types available to query.

```
-- schema.sql
```

```

CREATE TABLE matches (
  match_id INTEGER PRIMARY KEY,
  season TEXT,
  date TEXT,
  city TEXT,
  venue TEXT,
  team1 TEXT,
  team2 TEXT,
  toss_winner TEXT,
  toss_decision TEXT,
  winner TEXT,
  result TEXT,
  result_margin INTEGER,
  eliminator TEXT,
  umpire1 TEXT,
  umpire2 TEXT
);

CREATE TABLE deliveries (
  match_id INTEGER,
  inning INTEGER,
  batting_team TEXT,
  bowling_team TEXT,
  over INTEGER,
  ball INTEGER,
  batter TEXT,
  bowler TEXT,
  non_striker TEXT,
  batsman_runs INTEGER,
  extra_runs INTEGER,
  total_runs INTEGER,
  is_wicket INTEGER,
  dismissal_kind TEXT,
  player_dismissed TEXT,
  fielder TEXT,
  FOREIGN KEY(match_id) REFERENCES matches(match_id)
);

```

3. The Golden Dataset

A **Golden Dataset** is a curated list of representative user questions paired with their perfect, validated SQL queries. In a production environment, our **Data Analysts** must construct this dataset manually. For our evaluation, we curate a set of **20 exceptionally difficult (brutal) questions** containing complex joins, subqueries, and window functions to stress-test the models.

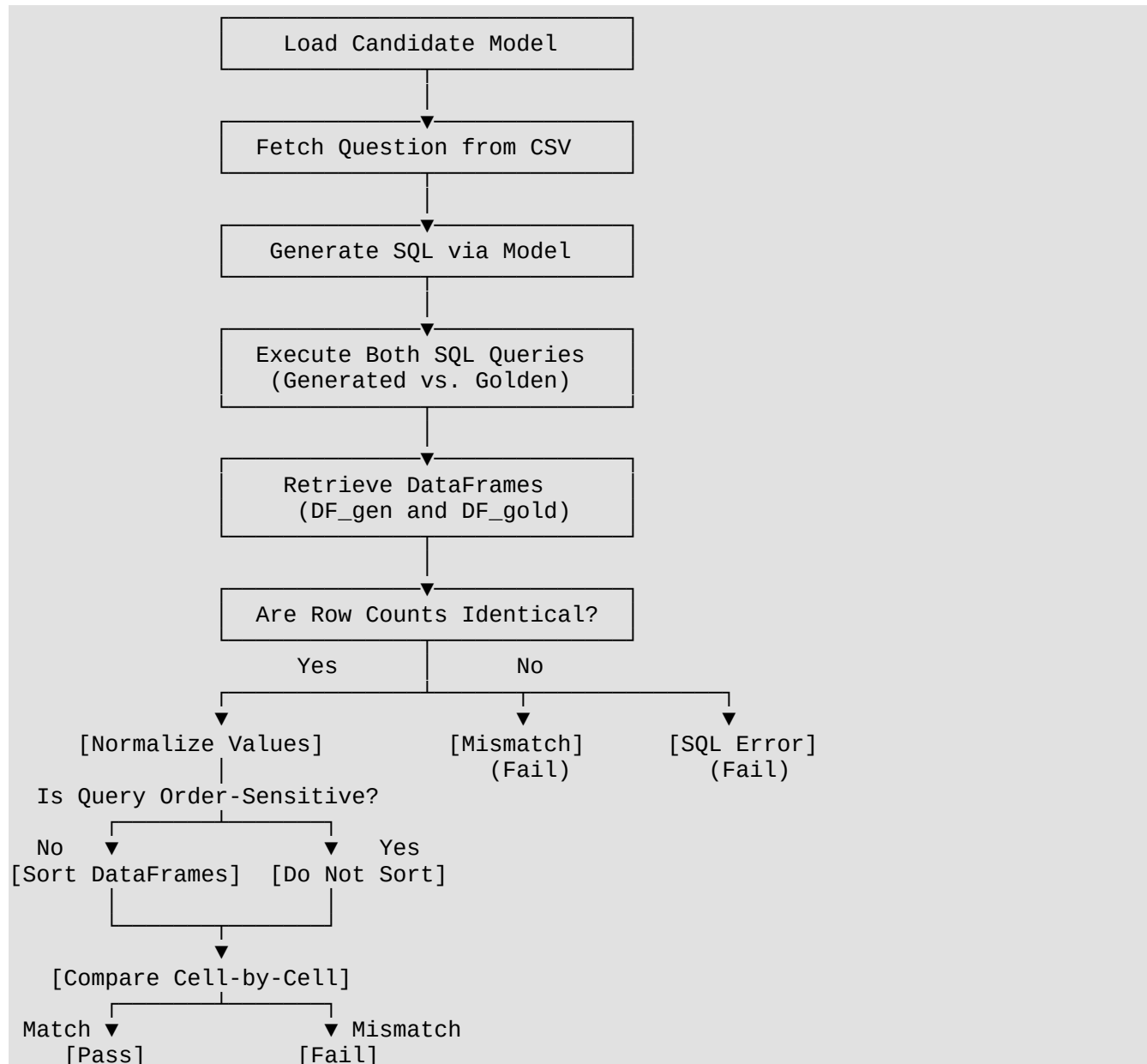
We write a validation script (`golden_data_set_generator.py`) to execute all 20 golden queries on our database to ensure they are 100% free of syntax errors and yield the correct mathematical results. Once validated, we run `make_golden_data_set.py` to compile these questions and queries into a structured CSV file (`golden_hard.csv`).

Crucially, this CSV also tracks two additional metadata fields:

- `expected_rows`: The exact number of rows the correct query must return.
- `order_sensitive`: A boolean flag indicating whether the query's correctness depends on a

specific sorting order (i.e., whether it utilizes an `ORDER BY` clause).

The Evaluation Pipeline and Comparison Logic



To run the evaluation, we construct an orchestrator file (`main.py`) and a specialized comparison script (`evaluator.py`).

Why Simple String Matching Fails

We cannot evaluate a model by performing a character-by-character string comparison between its generated SQL and our golden SQL. SQL is a highly expressive language; there are always multiple structurally distinct queries that are semantically identical and return the exact same data table.

Our Robust DataFrame Comparison Algorithm

Instead of comparing the code, we execute **both** the generated query and the golden query on our database, retrieve their outputs as Pandas DataFrames (`DF_gen` and `DF_gold`), and run them through our comparison algorithm in `evaluator.py`:

1. **Row Count Validation:** If the number of rows in `DF_gen` does not match `DF_gold`, we immediately log a **Mismatch (Fail)**. If the generated query fails to execute due to bad SQL syntax, we log a **SQL Error (Fail)**.
 2. **Data Type Normalization:** To prevent trivial mismatches, we normalize the cell values. For instance, we convert float representations (like `2.0`) to integers (`2`) to match the golden set, and handle minor floating-point rounding discrepancies (e.g., matching `2.999` to `2.99`).
 3. **Order Sensitivity Handling:**
 - **If `order_sensitive` is False:** We sort both DataFrames. Sorting is essential because different query planning approaches might return rows in a different order, which is still correct. Once sorted, we perform a cell-by-cell value comparison.
 - **If `order_sensitive` is True:** We bypass sorting. We compare the DataFrames exactly as they are returned, which verifies that the candidate model correctly implemented the required `ORDER BY` logic.
 4. **Final Verdict:** If all cells match, we record a **Match (Pass)**.
-

6. Live Evaluation Results and Analysis

We executed the complete evaluation pipeline using the unified **OpenRouter API** (which integrates cleanly with LangChain via `ChatOpenRouter`) to test all 5 candidate models against our 20 brutal questions. Here are the exhaustive results of the run:

Model	Accuracy (%)	Performance Classification & Core Observations
Grok	90%	The Clear Winner: Solved 18 out of 20 questions. It successfully resolved the highly complex Question #2 (which both GPT Tera and Kimi K3 failed). It was incredibly fast, had zero syntax errors, and is exceptionally cheap (\$2.50 per Million tokens blended, which is less than half the cost of GPT Tera).
Claude 3.5 Sonnet 5	85%	The Strongest Alternative: Solved 17 out of 20 questions. It

Model	Accuracy (%)	Performance Classification & Core Observations
		was subjectively the fastest model in the entire run. It generated highly clean, robust SQL with zero syntax errors.
GPT 5.6 Tera	80%	Underperformed relative to Cost: Solved 16 out of 20 questions. While highly capable, it failed on several difficult joins and is extremely expensive, pushing the absolute limit of our monthly budget.
MiniMax M3	65%	Poor Syntactic Reliability: Solved 13 out of 20 questions. While fast and incredibly cheap, it suffered from severe SQL syntax errors, occasionally outputting completely broken queries. This is a common issue with Chinese open-weight models on specialized coding tasks.
Kimi K3 (Km K3)	55%	The Biggest Disappointment: Solved only 11 out of 20 questions. Despite its massive 2.7-Trillion parameter architecture and heavy leaderboard hype, it performed terribly. It was extremely slow (due to its heavy reasoning steps), and generated 6 blatant SQL syntax errors.

The Biggest Disappointment: Solved only 11 out of 20 questions. Despite its massive 2.7-Trillion parameter architecture and heavy leaderboard hype, it performed terribly. It was extremely slow (due to its heavy reasoning steps), and generated 6 blatant SQL syntax errors.

Key Takeaway on Leaderboard Hype

Our results demonstrate a vital practical lesson: **A model's stellar performance on a public leaderboard is absolutely no guarantee of its success on your custom task.** Kimi K3 was hyped as a frontier class model, yet it failed miserably under the unique structural constraints of our custom Text-to-SQL database evaluation.

7. The Final Selection Decision

Based on our empirical metrics, our final choice narrows down to two models: **Grok** and **Claude 3.5 Sonnet 5**.

- **The Case for Grok:** It achieved the highest accuracy (90%) and is exceptionally cost-effective, easily fitting within our strict budget.
- **The Case for Claude 3.5 Sonnet 5:** It achieved 85% accuracy, but Anthropic's API infrastructure is highly stable and reliable. In contrast, Grok's provider environment can occasionally feel volatile or less reliable for enterprise production due to xAI's fast moving changes.
- **Our Decision:** In a corporate setting, if our engineering team cannot reach a consensus between Grok's cost-efficiency/accuracy and Sonnet's enterprise stability, we conduct a team vote. Both models are stellar candidates.